

PRODUCT BACKEND ENGINEER TEST

Concert Ticket Booking Platform

Backend System Design & Implementation Report

Engineering Focus

Correctness under concurrency

Idempotent retries •

Voucher consistency

Reservation lifecycle • Operational workflows

Testable engineering trade-offs

Candidate: Ticket Booking Backend Submission

Email: Not provided

Repository: Local repository submission

Version: 1.0

Date: August 23, 2026

Prepared for the Product Backend Engineer technical assessment.

Document Control

Document	Backend System Design & Implementation Report
Assessment	Product Backend Engineer Test — Concert Ticket Booking Platform
Candidate	Ticket Booking Backend Submission
Repository	Local repository submission
Version	1.0
Date	August 23, 2026

Contents

Document Control	i
1 Executive Summary	1
2 Requirements Analysis	3
2.1 Business Context	3
2.2 Actors and Trust Boundaries	4
2.3 Functional Requirements	4
2.4 Non-Functional Requirements	5
2.5 Use Case Diagram	6
2.6 Primary Engineering Risks and Design Goal	8
3 Scope, Assumptions, Business Rules, and Invariants	9
3.1 Scope Strategy	9
3.2 Implemented Scope	9
3.3 Assumptions	9
3.4 Business Rules	10
3.5 Critical Business Invariants	11
3.6 Non-Goals for the Assessment	12
4 System Architecture	13
4.1 Architecture Goals	13
4.2 System Context	13
4.3 Selected Architecture: Modular Monolith and PostgreSQL	15
4.4 Component and Module Architecture	16
4.5 Why PostgreSQL Is the Consistency Boundary	17
4.6 Alternatives and Trade-offs	18
4.7 Scaling Direction	18
5 Database Design	19
5.1 Data Model Goals	19

5.2	Core Entities	20
5.3	Entity Relationship Diagram	20
5.4	Critical Constraints	21
5.5	Index Strategy	22
5.6	Price Snapshotting	22
6	Booking Workflow and Transaction Boundary	23
6.1	Create Booking Sequence	23
6.2	Transaction Boundary	25
6.3	Failure Cases	25
6.4	Multi-Category Reservation	26
7	Concurrency and Overselling Prevention	27
7.1	Why Check-Then-Update Is Unsafe	27
7.2	Atomic Conditional Inventory Update	27
7.3	Alternatives Considered	28
7.4	Concurrency Strategy	29
7.5	Correctness Evidence	29
8	Idempotency and Retry Safety	30
8.1	Failure Scenario	30
8.2	API Contract	30
8.3	Database Enforcement	30
8.4	Request Fingerprint	31
8.5	Concurrent Retry Evidence	31
9	Voucher and Promotion Consistency	32
9.1	Voucher Validation	32
9.2	Atomic Global Quota Update	32
9.3	Per-User Anti-Abuse Rule	33
9.4	Release Semantics	33
9.5	Operation Scope	33
9.6	Correctness Evidence	33
10	Booking Lifecycle and Reservation Expiry	34
10.1	State Model	34
10.2	Why Reservation TTL Exists	35
10.3	Expiry Worker	35
10.4	Scalable Worker Claiming	36
10.5	Confirm-vs-Expire Race	37

11 API and Operation Workflows	38
11.1 Customer-Facing API Scope	38
11.2 Operation API Scope	39
11.3 Availability Semantics	40
11.4 Error Contract	40
12 Security, Reliability, and Observability	41
12.1 Security Boundary	41
12.2 Reliability Controls	41
12.3 Observability	42
12.4 Implementation Status and Evolution	42
13 Testing Strategy and Correctness Evidence	43
13.1 Testing Layers	43
13.2 Invariant-to-Test Evidence	44
13.3 Requirement Traceability	44
13.4 Evidence Standard	45
14 Performance and Flash-Sale Analysis	46
14.1 Expected Workload and Evaluation Goal	46
14.2 Executed Environment	46
14.3 Executed Scenarios	47
14.4 Measured Results	47
14.5 Interpretation and Limitations	47
14.6 Optimization Priorities	48
15 Architecture Decisions and Trade-offs	49
15.1 Summary of Major Decisions	49
15.2 What Was Deliberately Not Chosen	49
15.2.1 Microservices	50
15.2.2 Redis Distributed Lock for Inventory	50
15.2.3 Kafka as a Baseline Requirement	50
15.3 Decision Review and Revisit Triggers	50
16 Implemented Features, Omitted Features, and Limitations	51
16.1 Implemented Scope Matrix	51
16.2 Known Limitations and Evolution	52
16.3 Feature Scope: Voucher Operations	53
16.3.1 Assumption	53
16.3.2 Implemented	53

16.3.3 Not Implemented	53
16.3.4 Reason, Consequence, and Production Evolution	53
17 Payment Gateway Scope Decision	54
17.1 Current Assessment Decision	54
17.2 Implemented Payment-Adjacent Behavior	54
17.3 Explicitly Not Implemented	55
17.4 Why Production Uses an External Gateway	55
17.5 Transaction Design Rule	56
17.6 Limitation and Evolution Statement	56
18 Conclusion	57
A Local Setup and Runbook	58
A.1 Prerequisites	58
A.2 Quick Start	58
A.3 Seeded local identities	58
A.4 Verification	59
A.5 Test Commands	59
B API Endpoint Matrix	60
C Test Evidence Summary	61
C.1 Evidence Capture	61
D Architecture Decision Records	63
D.1 ADR-001 — Modular Monolith	63
D.2 ADR-002 — PostgreSQL for Inventory Consistency	63
D.3 ADR-003 — Atomic Conditional Inventory Update	63
D.4 ADR-004 — Idempotent Booking API	64
D.5 ADR-005 — Database-Polling Expiry Worker	64
D.6 ADR-006 — Transactional Outbox (Optional)	64

List of Figures

2.1	Current-scope actor and use case view	7
4.1	Current system context with explicitly out-of-scope future actors	14
4.2	Modular-monolith component view	16
5.1	Implemented PostgreSQL entity relationship model	21
6.1	Canonical idempotent create-booking sequence	24
10.1	Implemented booking lifecycle	34
10.2	Reservation expiry worker sequence and transaction boundary	36
17.1	Future external payment flow (not implemented)	55

List of Tables

1.1	Final correctness and performance evidence	2
2.1	Current-scope actors	4
2.2	Functional requirement catalogue	4
2.3	Non-functional requirements	5
2.4	Assessment risks and intended engineering response	8
3.1	Key assessment assumptions	10
3.2	Enforced business rules	10
4.1	Logical modules and responsibilities	17
4.2	Architecture alternatives considered	18
5.1	Core persistence model	20
5.2	Example query-to-index mapping	22
6.1	Create-booking failure semantics	25
7.1	Inventory concurrency alternatives	28
7.2	Concurrency hazards and controls	29
8.1	Idempotency behavior	31
10.1	Allowed lifecycle transitions	35
11.1	Representative customer endpoints	38
11.2	Representative operation endpoints	39
12.1	Security, reliability, and observability status	42
13.1	Testing strategy	43
13.2	Critical correctness evidence	44
13.3	Critical requirement traceability	44
14.1	Reproducible local load-test environment	46

15.1	Architecture trade-off summary	49
15.2	Operational triggers for architecture evolution	50
16.1	Implementation scope	51
17.1	Payment transaction boundary	56
B.1	Implemented API endpoint matrix	60
C.1	Critical automated test evidence	61

Listings

5.1	Representative database constraints	21
7.1	Atomic inventory reservation	27
8.1	Idempotent booking request	30
9.1	Atomic voucher quota reservation	32
10.1	Disjoint batch claim for expired reservations	36
10.2	Conditional confirmation	37
11.1	Example business error response	40
A.1	Fresh local setup	58
A.2	Docker Compose setup	58
A.3	Final quality gates	59

Executive Summary

The Concert Ticket Booking Platform serves customer reservation flows and internal operation workflows. Its principal risks are not raw CRUD complexity but contention over finite ticket inventory and voucher quota, duplicate bookings caused by network retries, invalid lifecycle transitions, and burst traffic during flash sales.

The submitted solution is a **modular monolith** exposing a stateless REST API on Node.js 22 and NestJS, with PostgreSQL 16 as the source of truth and consistency boundary. Booking, inventory, voucher, idempotency, operation, and expiry-worker responsibilities remain logically separated while sharing one local ACID transaction where correctness requires coordination. Docker Compose, versioned SQL migrations, deterministic seed data, OpenAPI, and Postman assets make the solution reproducible.

The critical booking path uses conditional inventory updates, deterministic resource ordering, database-backed idempotency with request fingerprints, atomic voucher quota reservation, server-side price calculation, immutable price snapshots, and a ten-minute **RESERVED** TTL. A polling worker claims expired rows with **FOR UPDATE SKIP LOCKED**; confirmation, cancellation, manual operation changes, and expiry all use conditional lifecycle writes so only one competing terminal transition can win.

Table 1.1: Final correctness and performance evidence

Evidence	Verified result
Automated tests	12 unit suites / 32 tests; 7 integration suites / 22 tests; 4 concurrency suites / 7 tests
Oversell race	100 attempts for 10 tickets produce exactly 10 bookings and final inventory zero
Retry and voucher races	One logical booking per idempotency key and at most one winner for the final voucher unit
Lifecycle race	Confirm versus expiry produces exactly one valid terminal state with no double release
k6 baseline	501 requests, 501 booking successes, 0.000% system errors, p95 24.87 ms and p99 31.73 ms
Delivery gates	Format, lint, typecheck, build, Newman smoke, submission and delivery verification pass in CI

The scope intentionally excludes real payment processing, seat-level selection, refunds, notifications, full catalog/voucher CRUD, and multi-region deployment. Confirmation is simulated or performed through an authorized current workflow. A future external payment provider would be called only after the reservation transaction commits; verified callbacks would drive an idempotent **RESERVED** -> **CONFIRMED** transition. These boundaries keep the assessment focused on demonstrable booking correctness without presenting future infrastructure as already implemented.

Requirements Analysis

2.1 Business Context

The platform supports two workflow groups. Customers browse published concerts, inspect category prices and availability, reserve finite inventory, optionally apply a voucher, and track a booking through a deliberately small lifecycle. Internal operators and administrators monitor bookings, investigate failures or suspicious activity, perform guarded status changes, and maintain the minimal concert/category/voucher setup implemented for the assessment. The assessment brief describes this context.

The assessment describes about 50,000 users and a peak of 300–500 booking requests per minute (approximately 5–8.3 requests per second). This throughput is modest for PostgreSQL; the difficult property is synchronized demand for the same last tickets or voucher units during a flash sale.

2.2 Actors and Trust Boundaries

Table 2.1: Current-scope actors

Actor	Primary goals	Trust / scope note
Customer	Browse, reserve, retry safely, view, confirm, and cancel own bookings	Identified by a seeded <code>x-user-id</code> ; this is an assessment simplification, not production authentication
Operator	Monitor and investigate bookings; perform permitted operation workflows	Must hold <code>OPERATOR</code> or <code>ADMIN</code> ; successful manual transitions are audited
Administrator	Same implemented operation capabilities as Operator	Separate business actor, currently sharing the same elevated route policy
System Worker	Expire overdue reservations and release finite resources	Internal background actor; no public endpoint or human identity

An external Payment Gateway is **not** a current actor because no payment integration exists. It appears only in the future/out-of-scope context in Chapter 17.

2.3 Functional Requirements

The source classification distinguishes requirements explicit in the brief, requirements derived from those workflows, and engineering additions required to preserve correctness. “Partial” means the implemented assessment workflow is intentionally narrower than full production CRUD or integration scope.

Table 2.2: Functional requirement catalogue

ID	Description	Actor	Source	Priority	Status
FR-01	Browse published concerts	Customer	Explicit	Must	Implemented
FR-02	View concert details	Customer	Explicit	Must	Implemented
FR-03	View ticket categories and server prices	Customer	Explicit	Must	Implemented
FR-04	View informational ticket availability	Customer	Explicit	Must	Implemented
FR-05	Create a time-limited ticket reservation	Customer	Explicit	Must	Implemented
FR-06	Apply an eligible promotional voucher	Customer	Explicit	Must	Implemented
FR-07	View owned booking detail and status	Customer	Derived	Must	Implemented
FR-08	View paginated own booking history	Customer	Explicit	Should	Implemented

ID	Description	Actor	Source	Priority	Status
FR-09	Monitor and filter bookings	Operator / Ad- min.	Explicit	Must	Implemented
FR-10	Inspect booking, voucher, idempotency, and audit detail	Operator / Ad- min.	Derived	Must	Implemented
FR-11	Perform a permitted manual booking transition with reason	Operator / Ad- min.	Explicit	Must	Implemented
FR-12	Create draft concerts and inventory-backed categories	Operator / Ad- min.	Explicit	Should	Implemented
FR-13	Publish only a valid future concert with inventory	Operator / Ad- min.	Derived	Should	Implemented
FR-14	Create a validated, optionally scoped voucher campaign	Operator / Ad- min.	Explicit	Should	Implemented
FR-15	Update/delete concert or voucher definitions	Operator / Ad- min.	Explicit	Optional	Not implemented
FR-16	Confirm or cancel a live owned reservation	Customer	Derived	Must	Implemented
FR-17	Expire overdue reservations automatically	System Worker	Eng. addition	Must	Implemented
FR-18	Release inventory after eligible cancellation/expiry	System Worker	Eng. addition	Must	Implemented
FR-19	Release reserved voucher quota exactly once	System Worker	Eng. addition	Must	Implemented
FR-20	Replay repeated booking requests idempotently	Customer / System	Eng. addition	Must	Implemented

2.4 Non-Functional Requirements

Table 2.3: Non-functional requirements

ID	Requirement	Acceptance evidence	Status
NFR-01	Inventory remains non-negative under contention	DB checks + CONC-001	Implemented
NFR-02	One logical request reserves resources at most once	Unique key/fingerprint + CONC-002	Implemented
NFR-03	Voucher quota and per-user policy remain consistent	Conditional update/index + CONC-003	Implemented
NFR-04	Booking creation is atomic across dependent writes	Transaction rollback integration tests	Implemented
NFR-05	Flash-sale failures are controlled, not silent corruption	Stable 4xx errors, limiter, k6 metrics	Implemented
NFR-06	Support the stated 300–500 booking requests/minute profile	501-request steady/burst k6 run	Demonstrated locally
NFR-07	Concurrent lifecycle commands produce one valid final state	Conditional writes + CONC-004	Implemented
NFR-08	Enforce ownership, operation roles, validation, and safe config	API/integration/security checks	Assessment-grade
NFR-09	Preserve maintainable domain/module boundaries	Modular monolith structure + ADR-001	Implemented
NFR-10	Permit deterministic automated verification	Unit, integration, concurrency, Postman, and k6	Implemented

ID	Requirement	Acceptance evidence	Status
NFR-11	Expose correlation, health, structured logs, and metrics	Request ID, probes, JSON logs, <code>/metrics</code>	Implemented locally
NFR-12	Reproduce setup from a clean clone	Lockfile, Compose, migrations, seed, runbook	Implemented

2.5 Use Case Diagram

Figure 2.1 is limited to implemented actors and capabilities. Internal include/extend relationships expose the correctness mechanisms that support actor goals without turning them into separate external actors.

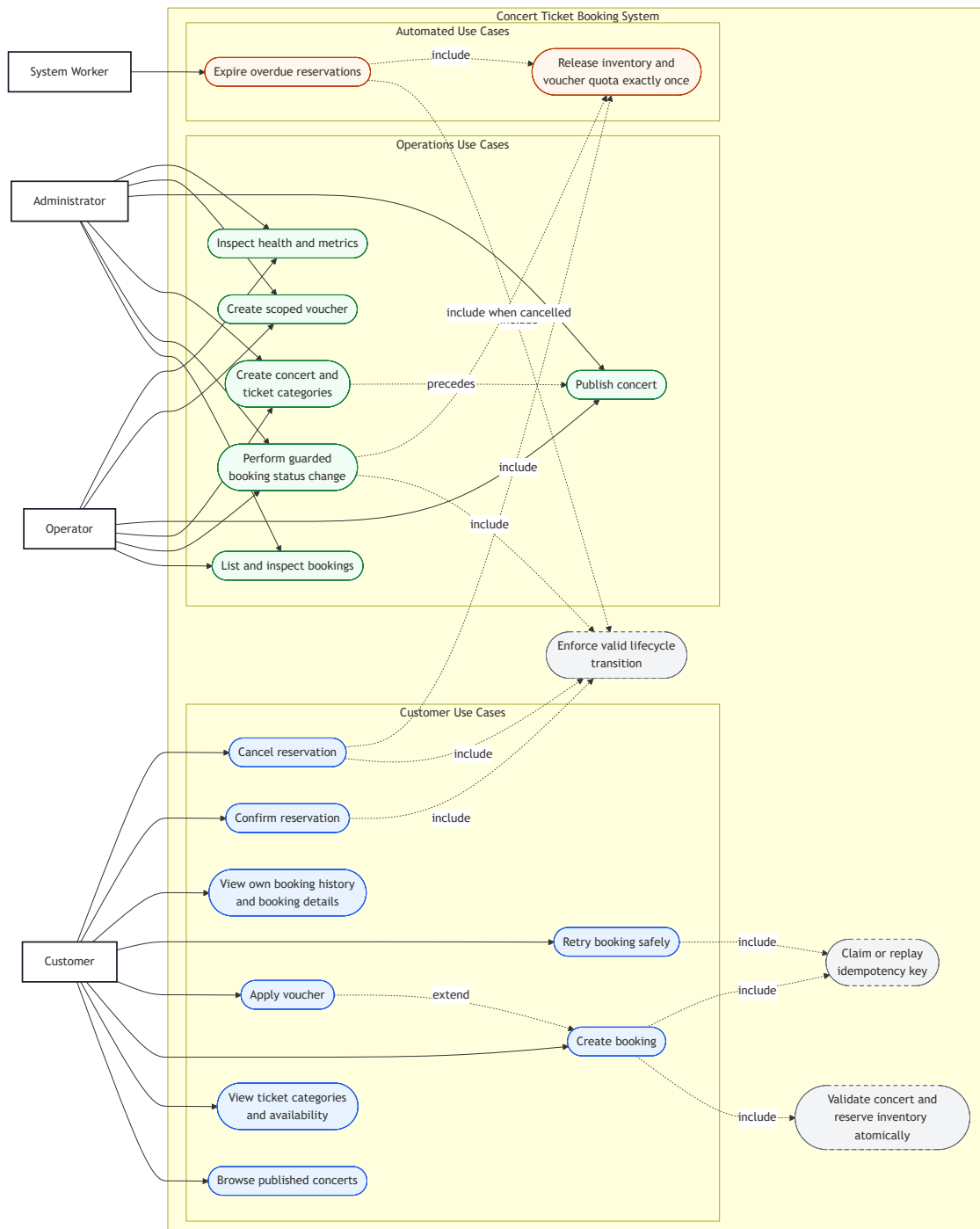


Figure 2.1: Current-scope actor and use case view

2.6 Primary Engineering Risks and Design Goal

Table 2.4: Assessment risks and intended engineering response

Risk	Failure mode	Design response
Overselling	More tickets sold than inventory	Atomic conditional inventory update inside a DB transaction
Duplicate retries	Multiple bookings from one user intent	Idempotency key, uniqueness, fingerprint, and response replay
Voucher abuse	Global/per-user quota exceeded	Atomic quota update and redemption constraint
Flash-sale instability	Retry storms and hot-row contention	Short transactions, controlled errors, rate limiting, and measured load evidence

The design goal is a backend whose critical invariants remain correct under concurrent and retried requests, with enough explicit evidence that a reviewer can reproduce and defend each decision. Feature count is secondary to that correctness objective.

Scope, Assumptions, Business Rules, and Invariants

3.1 Scope Strategy

The assessment explicitly permits a reasonable, documented feature boundary. The implementation therefore prioritizes customer booking correctness, a defensible minimal operation workflow, deterministic local delivery, and evidence under races. Production breadth is not claimed where the code does not provide it.

3.2 Implemented Scope

The submitted backend implements published-concert browse/detail/category reads; informational availability; transactional booking creation; optional voucher application; idempotent replay; owned booking history/detail; confirm/cancel commands; automatic expiry; operator booking list/detail/manual transition; draft concert/category creation and publication; scoped voucher creation; stable errors; health/metrics/OpenAPI endpoints; and audit history.

Full update/delete administration, real payment, notification delivery, seat-level selection, refunds, and distributed deployment are deliberately excluded. Chapter 17 records the payment boundary in detail.

3.3 Assumptions

Table 3.1: Key assessment assumptions

ID	Assumption	Reason / consequence
A-01	Inventory is capacity-based by ticket category	Seat maps and exact-seat holds are outside the model
A-02	A new booking enters RESERVED	Resources are held before confirmation
A-03	Reservation TTL defaults to ten minutes and is configurable	Abandoned reservations cannot hold finite resources forever
A-04	Only published, future/bookable concerts accept bookings	Draft or invalid catalog data cannot consume stock
A-05	Prices and discounts are calculated only by the backend	Client totals are never authoritative
A-06	Booking items and totals are immutable historical snapshots	Later catalog/promotion changes do not rewrite history
A-07	Voucher status, window, scope, quota, and per-user rules are backend policies	The booking request supplies only an optional code
A-08	Authentication is simplified to a seeded <code>x-user-id</code>	Ownership and RBAC are enforced, but JWT/OIDC/token verification is not claimed
A-09	Real payment processing is outside assessment scope	No card, wallet, provider, settlement, refund, or webhook flow exists
A-10	Confirmation is simulated through an owned customer command or authorized operation transition	CONFIRMED means accepted by the assessment workflow, not proven settlement

3.4 Business Rules

Table 3.2: Enforced business rules

ID	Rule
BR-01	Every requested ticket quantity is a positive integer.
BR-02	Every ticket category belongs to the selected concert.
BR-03	A reservation succeeds only when sufficient inventory exists at the atomic write boundary.

ID	Rule
BR-04	A voucher is usable only while active and inside its validity window.
BR-05	Voucher usage cannot exceed its global <code>usage_limit</code> .
BR-06	A user cannot hold more than one non-released redemption for the same voucher.
BR-07	Subtotal, discount, and final amount are calculated by the server from persisted policies.
BR-08	Persisted booking price snapshots do not change when current prices or voucher definitions change.
BR-09	<code>RESERVED</code> is the only mutable booking state; terminal transitions must pass their command-specific time guard.
BR-10	Eligible cancellation/expiry releases inventory and reserved voucher quota exactly once in the winning transition transaction.

3.5 Critical Business Invariants

Business Invariant INV-01 — Inventory safety

For every category, $0 \leq \text{available_quantity} \leq \text{total_quantity}$ always holds, including under concurrent booking attempts.

Business Invariant INV-02 — Exactly-once logical reservation

One user-scoped logical booking intent consumes inventory at most once, even if the client retries concurrently or after losing a response.

Business Invariant INV-03 — Voucher quota safety

Global voucher usage never exceeds its configured quota and the active per-user redemption constraint is never violated.

Business Invariant INV-04 — Atomic booking creation

Idempotency claim, inventory reservation, voucher reservation, server pricing, booking/items, audit history, and replayable result either all commit or all roll back as implemented by the critical transaction.

Business Invariant INV-05 — Exactly-once resource release

Expired or cancelled reservations restore ticket and eligible voucher capacity exactly once; repeat worker/customer/operator commands cannot double-release.

Business Invariant INV-06 — Historical price integrity

Changing current ticket or promotion data cannot alter an existing booking's persisted unit prices or monetary totals.

Business Invariant INV-07 — Lifecycle validity

Every status change follows RESERVED -> CONFIRMED | EXPIRED | CANCELLED; terminal states cannot be overwritten by a racing command.

Business Invariant INV-08 — Operation auditability

Every successful creation or manual/worker lifecycle transition appends status history with source, reason where applicable, actor when available, and time.

3.6 Non-Goals for the Assessment

- real payment authorization, capture, settlement, reconciliation, refunds, or chargebacks;
- seat-map / exact-seat selection;
- voucher or concert update/delete workflows beyond implemented creation/publication;
- email/SMS delivery and real-time WebSocket availability;
- multi-region active-active deployment or full microservice decomposition;
- mandatory Redis, Kafka, or Kubernetes as correctness prerequisites.

System Architecture

4.1 Architecture Goals

The architecture must keep correctness-sensitive transactions short and understandable, preserve strong consistency for finite resources, support the stated workload without speculative infrastructure, remain maintainable and testable, reproduce locally, and expose explicit evolution seams.

4.2 System Context

Figure 4.1 distinguishes current dependencies from future external systems. PostgreSQL is the only current external consistency service. Payment and notification providers are deliberately dashed and labelled as future/out of scope.

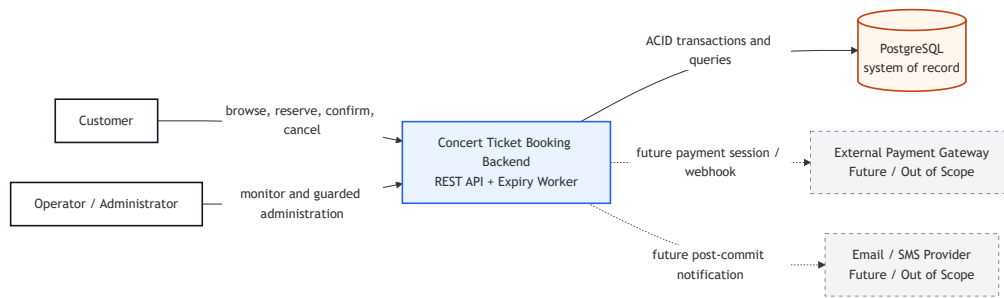


Figure 4.1: Current system context with explicitly out-of-scope future actors

4.3 Selected Architecture: Modular Monolith and PostgreSQL

Decision Use a modular monolith for the assessment

Booking, inventory, voucher, and idempotency require strong transactional coordination. Keeping these modules in one deployable process and one database avoids distributed transactions while retaining explicit domain boundaries. The stateless API can still be replicated horizontally when measured demand requires it.

This selection is proportional to the stated 5–8.3 booking requests per second. Microservices would add network failure modes, service discovery, cross-service retry semantics, and distributed coordination without improving the finite-resource guarantees demonstrated by this submission.

4.4 Component and Module Architecture

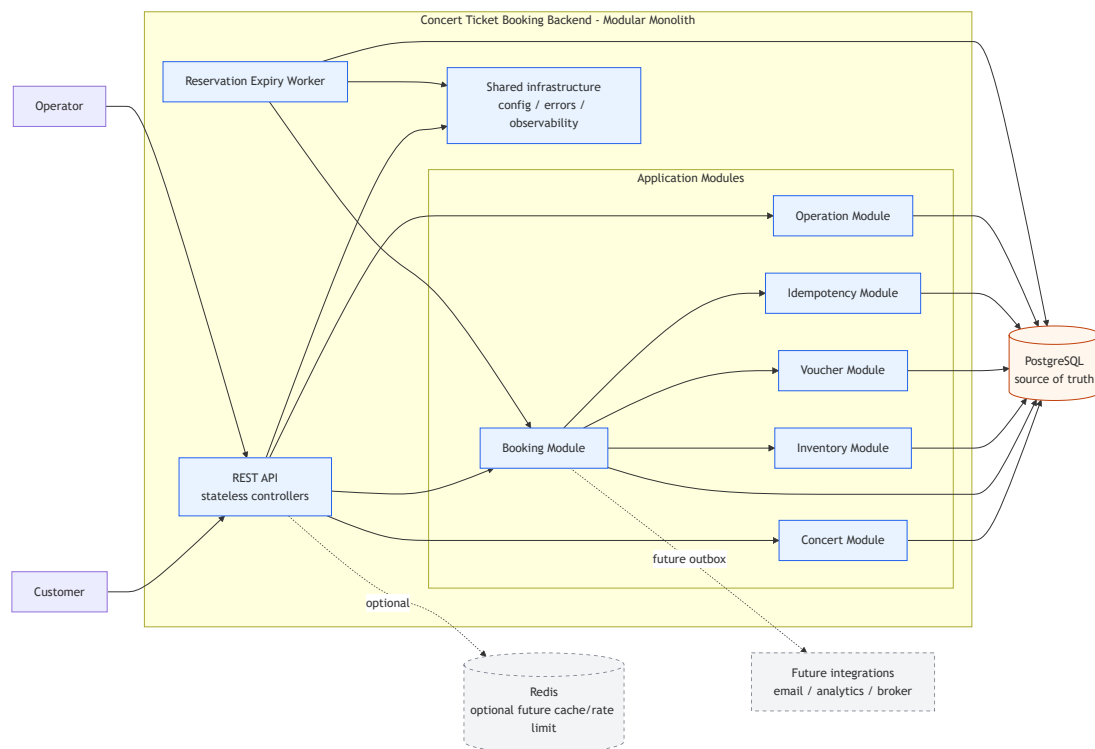


Figure 4.2: Modular-monolith component view

Table 4.1: Logical modules and responsibilities

Module	Responsibility
Concert	Published catalog, categories, prices, and informational availability
Inventory	Atomic ticket reservation and exactly-once release primitives
Booking	Aggregate creation/read, server pricing, transaction orchestration, and lifecycle
Voucher	Eligibility, scope, discount policy, quota reservation/consume/release
Idempotency	Request fingerprint, user/key claim, conflict, and response replay
Operation	Elevated booking queries, guarded manual transitions, concert/category/voucher creation
Expiry Worker	Batch claim of overdue reservations and transactional resource release
Shared	Configuration, database/transaction helpers, validation, errors, rate limit, and observability

The dependency direction is **Controller -> Application/Use Case -> Domain and Repository ports -> Infrastructure**. Controllers translate HTTP concerns; they do not own business transactions.

4.5 Why PostgreSQL Is the Consistency Boundary

Ticket inventory, voucher quota, booking state, audit history, and idempotency records are relational and participate in the same business outcomes. PostgreSQL conditional writes, row locks, constraints, and ACID rollback make it the natural arbiter. Redis may later support a shared limiter/cache, but it is not authoritative inventory and no Redis lock is needed for correctness.

4.6 Alternatives and Trade-offs

Table 4.2: Architecture alternatives considered

Decision point	Alternative	Reason not selected now
Deployment boundary	Microservices	Cross-service consistency and operational overhead exceed the assessment need
Inventory coordination	Redis distributed lock	Adds a second failure/authority system around data already owned by PostgreSQL
Critical workflow	Eventual event-driven reservation	Cannot provide the same simple all-or-nothing booking result without larger saga semantics
Expiry processing	Broker-delivered delayed messages	Reliable broker scheduling is useful at scale but unnecessary for the current bounded polling workload

4.7 Scaling Direction

Scale stateless API replicas first only after measuring database contention. Multiple expiry workers already coordinate through `FOR UPDATE SKIP LOCKED`. If post-commit integrations grow, a transactional outbox can bridge database state to a broker without dual writes. Hot categories may ultimately need admission control, queueing, or inventory partitioning, but those are revisit decisions based on observed lock waits rather than baseline requirements.

Database Design

5.1 Data Model Goals

The schema must make finite-resource correctness enforceable in PostgreSQL, support short transactions, retain historical monetary values, provide an auditable booking lifecycle, and serve ownership/operation/expiry queries with purpose-built indexes.

5.2 Core Entities

Table 5.1: Core persistence model

Table	Purpose
<code>users</code>	Customer/operator identity used by the assessment implementation
<code>concerts</code>	Concert metadata and publication lifecycle
<code>ticket_categories</code>	VIP/Standard/etc. category and current sale price
<code>ticket_inventories</code>	Total and available capacity per ticket category
<code>bookings</code>	Booking aggregate, status, total amounts, expiry timestamp
<code>booking_items</code>	Quantity and immutable unit-price snapshots
<code>vouchers</code>	Promotion policy, validity period, global quota
<code>voucher_ redemptions</code>	Per-user voucher usage and booking linkage
<code>idempotency_ records</code>	Logical request deduplication and response replay state
<code>booking_status_ history</code>	Operational audit trail for transitions

5.3 Entity Relationship Diagram

Figure 5.1 shows the implemented schema. A transactional `outbox_events` table is intentionally absent because the outbox remains an ADR-backed future option rather than submitted runtime behavior.

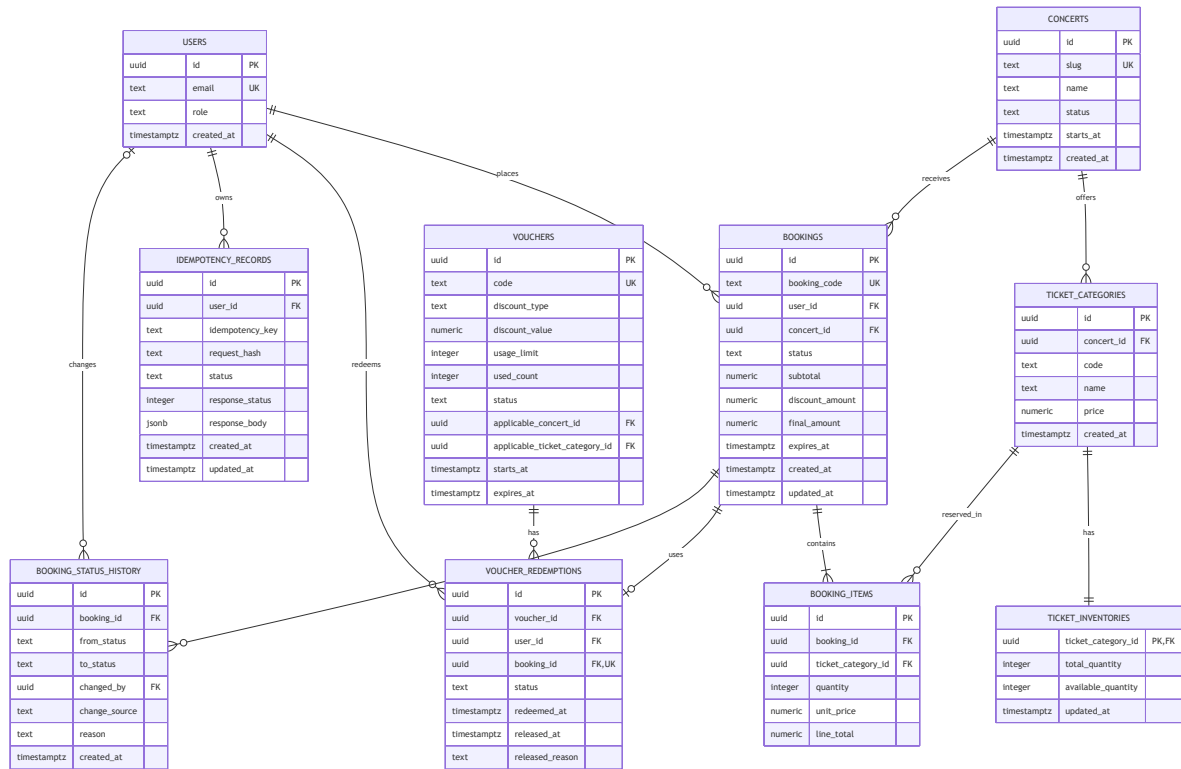


Figure 5.1: Implemented PostgreSQL entity relationship model

5.4 Critical Constraints

```

1 ALTER TABLE ticket_inventories
2   ADD CONSTRAINT chk_inventory_non_negative

```

```

3  CHECK (available_quantity >= 0),
4  ADD CONSTRAINT chk_inventory_not_above_total
5  CHECK (available_quantity <= total_quantity);
6
7  CREATE UNIQUE INDEX uq_idempotency_user_key
8  ON idempotency_records(user_id, idempotency_key);
9
10 CREATE UNIQUE INDEX uq_voucher_user
11 ON voucher_redemptions(voucher_id, user_id)
12 WHERE status <> 'RELEASED';

```

Listing 5.1: Representative database constraints

5.5 Index Strategy

Indexes should follow real query patterns rather than blanket indexing.

Table 5.2: Example query-to-index mapping

Query Pattern	Suggested Index
Expire old reservations	bookings(status, expires_at)
Customer booking history	bookings(user_id, created_at)
Operation filter by concert/status	bookings(concert_id, status, created_at)
Voucher lookup by code	unique vouchers(code)
Idempotency replay	unique idempotency_records(user_id, idempotency_key)

5.6 Price Snapshotting

The request never supplies an authoritative total. The server reads current ticket price and promotion policy, calculates the amount, and persists immutable snapshots in `booking_items` and `bookings`. This prevents future catalog changes from rewriting booking history.

Booking Workflow and Transaction Boundary

6.1 Create Booking Sequence

The create-booking use case converts one authenticated customer intent into one replayable result. Figure 6.1 shows the new-key, same-payload replay, changed-payload conflict, sold-out, and voucher-failure branches.

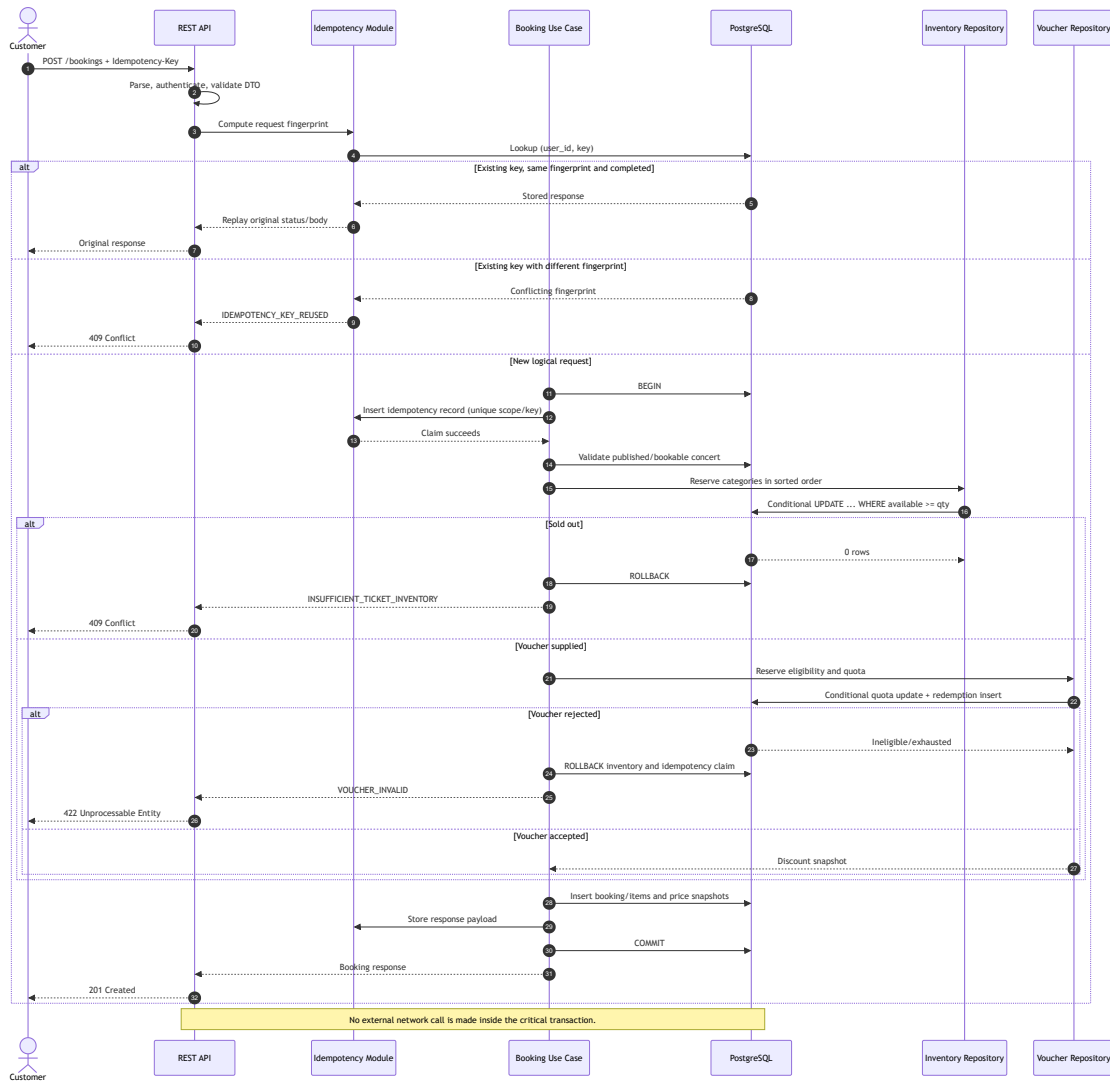


Figure 6.1: Canonical idempotent create-booking sequence

The implemented reasoning sequence is:

1. authenticate the assessment identity, validate the DTO, normalize items, and calculate a canonical request fingerprint;

2. inspect an already-completed idempotency record for an immediate replay/conflict path;
3. for a new logical request, begin one database transaction and claim the user, key, and fingerprint;
4. validate that the concert and categories are bookable;
5. reserve every category by atomic conditional update in deterministic order;
6. validate and reserve voucher policy/quota when supplied;
7. calculate subtotal, discount, and final amount from persisted server data;
8. insert the booking, immutable items, initial status history, and completed idempotency response;
9. commit and return the result.

6.2 Transaction Boundary

The idempotency claim/completion, ticket reservation, optional voucher quota and redemption, price snapshots, booking/items, and initial audit history share one transaction. A failure in any dependent operation rolls all of them back. This avoids a completed idempotency record for a booking that never committed and avoids inventory or voucher leakage after later validation fails.

External calls such as email, analytics, or a future payment provider must not extend this database transaction. They occur only after commit or through a reliable post-commit pattern such as the optional transactional outbox.

6.3 Failure Cases

Table 6.1: Create-booking failure semantics

Failure	API behavior	Database outcome
Invalid/unpublished concert or category	Controlled validation/business error	No resource or booking mutation
Insufficient inventory	INSUFFICIENT_TICKET_INVENTORY conflict	Whole request rolls back; stock remains valid
Invalid/ineligible/exhausted voucher	Stable voucher error	Earlier inventory writes roll back
Same key, same fingerprint	Original status/body replayed	No second booking or decrement
Same key, changed fingerprint	IDEMPOTENCY_KEY_REUSED conflict	Existing logical result remains untouched
Unexpected database error	Sanitized server error	Transaction rollback; no partial success is stored

6.4 Multi-Category Reservation

Category identifiers are normalized and processed in deterministic order. The consistent lock/update order reduces deadlock risk, while one transaction makes the request all-or-nothing: failure to reserve any category automatically rolls back every earlier category and optional voucher write.

Concurrency and Overselling Prevention

7.1 Why Check-Then-Update Is Unsafe

A sequence that first reads `available_quantity` and later updates it can oversell because concurrent transactions may observe the same pre-update value.

7.2 Atomic Conditional Inventory Update

```
1 UPDATE ticket_inventories
2 SET available_quantity = available_quantity - :quantity
3 WHERE ticket_category_id = :ticket_category_id
4    AND available_quantity >= :quantity
5 RETURNING available_quantity;
```

Listing 7.1: Atomic inventory reservation

A returned row means the reservation succeeded. No returned row means there was insufficient inventory. The correctness decision is made by PostgreSQL rather than by a stale application read.

7.3 Alternatives Considered

Table 7.1: Inventory concurrency alternatives

Option	Correctness potential	po-	Assessment trade-off
Application read then update	Unsafe		Race window exists between the read and write
Pessimistic row lock	Correct		Explicit lock/read/write path is longer than the selected one-statement predicate
Optimistic version column	Correct with retry		Adds version/retry policy for a resource naturally guarded by quantity predicate
Atomic conditional update	Correct; selected		PostgreSQL decides against current row state in one statement
Redis distributed lock	Potentially correct with careful failure handling		Creates a second coordination authority and lock-expiry failure modes
Queue serialization	Correct but asynchronous	asyn-	Adds queue latency/operations and changes synchronous request semantics

7.4 Concurrency Strategy

Table 7.2: Concurrency hazards and controls

Hazard	Control
Two customers compete for final ticket	Atomic conditional update
Booking contains multiple categories	Deterministic category order + one DB transaction
Voucher and inventory have different outcomes	Same transaction; any failure rolls back both
Admin/worker changes stale booking state	Conditional status update based on current state
Multiple expiry workers	Row claim with <code>FOR UPDATE SKIP LOCKED</code> (optional plus point)

7.5 Correctness Evidence

CONC-001 seeds 10 tickets and launches 100 simultaneous booking attempts. The verified result is exactly 10 successful bookings, 90 controlled conflicts, final inventory zero, and no negative intermediate/final state. This checks persisted bookings and inventory rather than relying only on response codes.

Idempotency and Retry Safety

8.1 Failure Scenario

A booking can be committed successfully while the HTTP response is lost. The client then retries because it cannot know whether the original request succeeded. Without idempotency, the retry may reserve inventory again and create a second booking.

8.2 API Contract

```
1 POST /bookings HTTP/1.1
2 Idempotency-Key: 44a3b9d0-...
3 Content-Type: application/json
4
5 {
6   "concertId": 1,
7   "items": [
8     { "ticketCategoryId": 10, "quantity": 2 }
9   ],
10  "voucherCode": "FLASH20"
11 }
```

Listing 8.1: Idempotent booking request

8.3 Database Enforcement

The idempotency key is scoped by authenticated user and protected by a unique constraint. The record stores a canonical request fingerprint and enough result data to identify/replay the completed logical booking.

8.4 Request Fingerprint

The canonical request can include user ID, concert ID, sorted ticket items, quantities, and normalized voucher code. A cryptographic hash (for example, SHA-256) is stored with the key.

Table 8.1: Idempotency behavior

Situation	Behavior
New key	Process the booking and persist logical result
Same key + same fingerprint	Return the original logical booking/result
Same key + different fingerprint	Reject with an idempotency conflict
Concurrent requests with same key	Exactly one request claims the logical operation; others replay/wait/reload according to implementation policy

8.5 Concurrent Retry Evidence

CONC-002 sends 20 concurrent requests with the same customer, idempotency key, and payload. The final database state contains one logical booking and one inventory decrement; every completed retry resolves to that same result. A companion integration path verifies that reusing the key with a different fingerprint returns a stable conflict and performs no new mutation.

Voucher and Promotion Consistency

9.1 Voucher Validation

Voucher validation includes status, validity window, optional concert/category scope, remaining global quota, and one-use-per-user policy. A global voucher has NULL scope columns; a scoped voucher must match the booking concert and at least one requested category. The server calculates the discount; the client cannot submit an authoritative discount amount.

9.2 Atomic Global Quota Update

```
1 UPDATE vouchers
2 SET used_count = used_count + 1
3 WHERE id = :voucher_id
4     AND status = 'ACTIVE'
5     AND starts_at <= NOW()
6     AND expires_at > NOW()
7     AND used_count < usage_limit
8 RETURNING used_count;
```

Listing 9.1: Atomic voucher quota reservation

The conditional update and redemption insert occur inside the same transaction as inventory and booking persistence. A voucher rejection therefore cannot leave a ticket reservation behind, and a later booking failure cannot consume promotion quota.

9.3 Per-User Anti-Abuse Rule

For a one-use-per-user policy, a partial unique index on (`voucher_id`, `user_id`) where redemption status is not `RELEASED` provides a database-backed guarantee. A retry of the same logical booking does not create another redemption because idempotency is applied before the booking can be duplicated.

9.4 Release Semantics

Voucher quota is reserved at booking creation, represented by a `RESERVED` redemption, and counted in `used_count`. Confirmation transitions it to `CONSUMED`. Expiration/cancellation transitions only a `RESERVED` redemption to `RELEASED`, decrements the quota in the same transaction, and keeps the audit row. Repeating release affects zero rows, so the allocation cannot be decremented twice.

9.5 Operation Scope

The elevated API supports creating an active, time-bounded percentage or fixed voucher with optional concert/category scope. Update and delete endpoints are not implemented: changing a policy already referenced by historical or active redemptions requires explicit versioning/audit rules beyond this assessment.

9.6 Correctness Evidence

CONC-003 has 20 users compete for one remaining global voucher unit and verifies exactly one successful redemption and a quota that never exceeds its limit. Integration tests also prove eligibility/scope errors roll back ticket inventory and that release preserves the redemption audit row while decrementing `used_count` only once.

Booking Lifecycle and Reservation Expiry

10.1 State Model

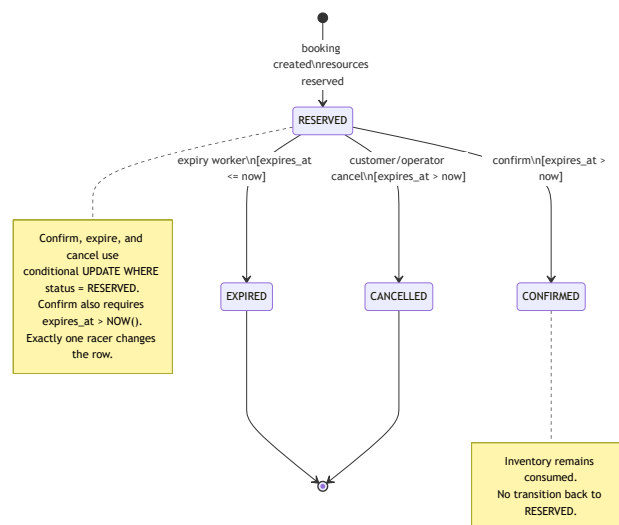


Figure 10.1: Implemented booking lifecycle

Table 10.1: Allowed lifecycle transitions

From	To	Allowed	Guard / effect
RESERVED	CONFIRMED	Yes	Must still be live; resources remain consumed
RESERVED	CANCELLED	Yes	Customer requires live reservation; release resources once
RESERVED	EXPIRED	Yes	Must be past expiry; release resources once
CONFIRMED	Any state	No	Terminal in current scope
EXPIRED	Any state	No	Terminal in current scope
CANCELLED	Any state	No	Terminal in current scope

10.2 Why Reservation TTL Exists

Atomic creation prevents immediate overselling, but a reservation that never confirms must not hold finite resources forever. New bookings therefore store `status = RESERVED` and a configurable `expires_at` (ten minutes by default). TTL separates a short reservation transaction from later confirmation and bounds abandoned capacity.

10.3 Expiry Worker

The worker wakes every 30 seconds by default, prevents overlapping ticks in one process, requests a default batch of 100, and delegates one transactional batch to the lifecycle service. The service clamps externally supplied batch sizes to 1–500. Figure 10.2 shows the implemented flow.

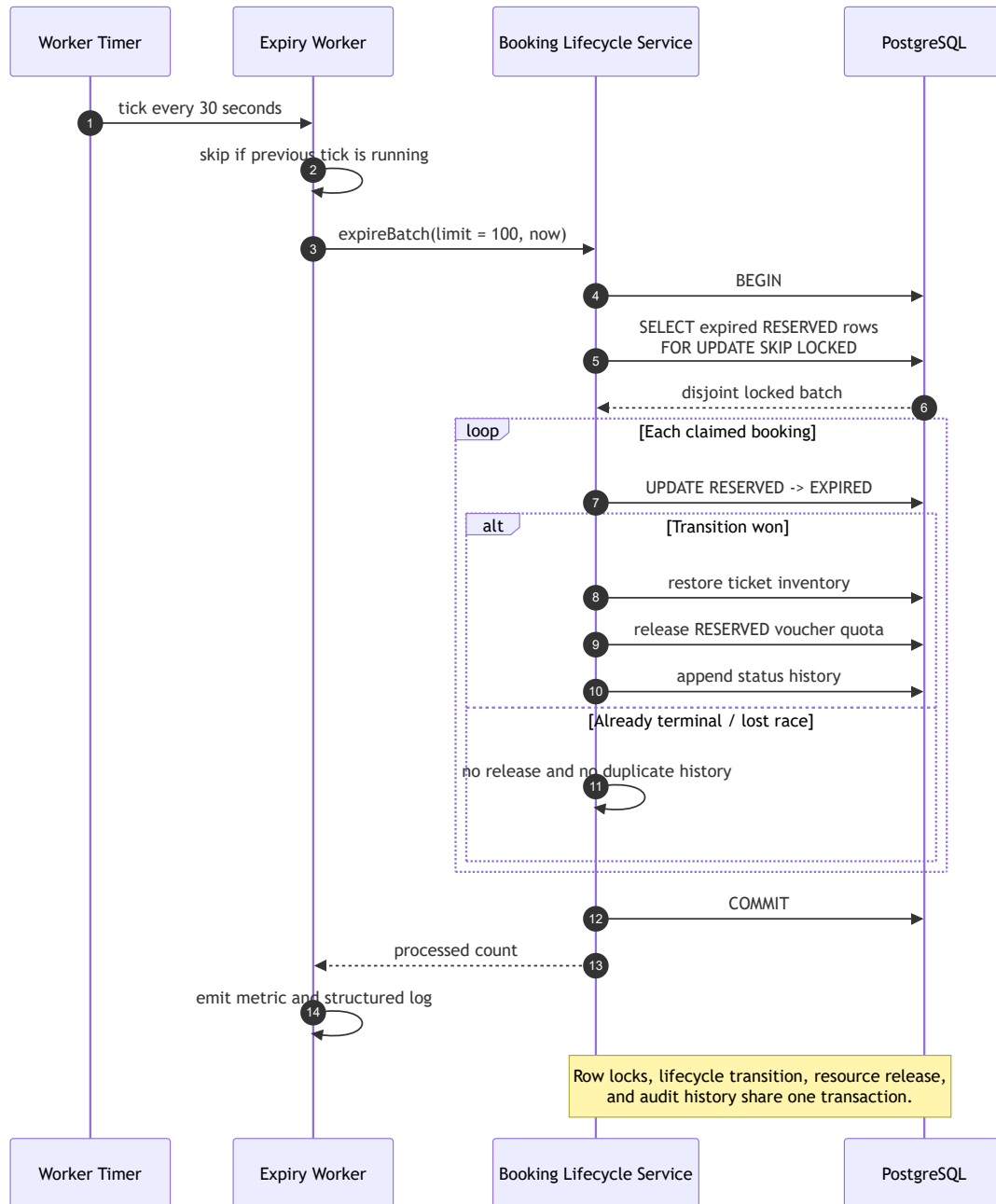


Figure 10.2: Reservation expiry worker sequence and transaction boundary

10.4 Scalable Worker Claiming

```

1 SELECT id
2 FROM bookings

```

```
3 WHERE status = 'RESERVED'
4     AND expires_at <= NOW()
5 ORDER BY expires_at, id
6 LIMIT :safe_limit
7 FOR UPDATE SKIP LOCKED;
```

Listing 10.1: Disjoint batch claim for expired reservations

Multiple workers claim disjoint rows without a distributed lock. Each claim lock is held until the conditional status update, ticket restoration, reserved voucher release, and audit insert commit. A repeat run cannot find the row as **RESERVED** and therefore cannot release resources twice.

10.5 Confirm-vs-Expire Race

```
1 UPDATE bookings
2 SET status = 'CONFIRMED', updated_at = NOW()
3 WHERE id = :booking_id
4     AND status = 'RESERVED'
5     AND expires_at > NOW();
```

Listing 10.2: Conditional confirmation

The expiry and confirmation paths both require **status = RESERVED**; the database row update is the arbitration point. The expiry/time predicate is evaluated after any lock wait, so stale application time cannot confirm an already-expired reservation. **CONC-004** proves one terminal outcome and a resource state consistent with that winner.

API and Operation Workflows

11.1 Customer-Facing API Scope

Table 11.1: Representative customer endpoints

Method	Endpoint	Purpose
GET	/api/concerts	Browse published concerts
GET	/api/concerts/{id}	Concert details
GET	/api/concerts/{id}/ ticket-categories	Categories, price, informational availability
POST	/api/bookings	Atomic reservation with optional voucher and idempotency key
GET	/api/bookings/{id}	Ownership-scoped booking detail/status
GET	/api/me/bookings	Paginated customer booking history
POST	/api/bookings/{id}/ confirm	Confirm a live owned reservation
POST	/api/bookings/{id}/ cancel	Cancel and release a live owned reservation

11.2 Operation API Scope

Table 11.2: Representative operation endpoints

Method	Endpoint	Purpose
GET	/api/admin/bookings	Monitor/filter bookings
GET	/api/admin/bookings/{id}	Investigate booking and audit detail
PATCH	/api/admin/bookings/{id}/status	Guarded, reasoned manual transition
POST	/api/admin/concerts	Create draft concert
POST	/api/admin/concerts/{id}/ticket-categories	Create category and initial inventory
POST	/api/admin/concerts/{id}/publish	Publish a valid future concert
POST	/api/admin/vouchers	Create a validated scoped/global voucher

Operation routes require an elevated operator/admin identity. Booking list filters are status, concert, user, and creation-time ranges with stable pagination. Booking detail exposes status history, voucher redemption state, and idempotency outcome timestamps without request hashes or stored response bodies.

Manual status commands accept only **CONFIRMED**, **EXPIRED**, or **CANCELLED** as targets from **RESERVED**. The command-specific expiry guard is evaluated in PostgreSQL; cancellation/expiry release inventory and voucher quota in the same transaction, and every successful command appends an **OPERATION_API** audit row with the required reason.

The intentionally minimal catalog scope creates draft concerts, creates ticket categories with initial inventory, publishes only future concerts with inventory, and creates validated vouchers. Update/delete APIs are omitted so published prices, inventory history, and already-redeemed voucher policy cannot be changed without a separately specified business rule.

11.3 Availability Semantics

Availability returned by read APIs is informational and may become stale immediately under concurrent demand. The authoritative inventory decision always occurs inside `POST/api/bookings`.

11.4 Error Contract

```
1 {  
2   "code": "INSUFFICIENT_TICKET_INVENTORY",  
3   "message": "Requested ticket quantity is no longer available",  
4   "details": {  
5     "ticketCategoryId": 10  
6   },  
7   "traceId": "..."  
8 }
```

Listing 11.1: Example business error response

Business conflicts should produce stable, documented errors rather than generic HTTP 500 responses.

Security, Reliability, and Observability

12.1 Security Boundary

Every customer or operation request requires a syntactically valid `x-user-id` that exists in the seeded `users` table. Customer routes require `CUSTOMER`; `/api/admin/**` requires `OPERATOR` or `ADMIN`. Booking reads and customer commands include the customer ID in the lookup predicate, so another customer's booking is not disclosed.

This header is assessment-grade identity injection, not authentication. A production deployment must derive the same subject/role from a verified JWT, OIDC provider, or trusted gateway. Existing ownership and RBAC checks remain necessary after that replacement.

Validation pipes reject malformed DTOs and non-whitelisted fields. The booking service rejects unpublished concerts, cross-concert categories, invalid quantities, untrusted price inputs, and invalid voucher policy. Configuration comes from environment variables; no runtime secret or private key is committed. Errors never expose stack traces, SQL, request hashes, or stored replay bodies.

12.2 Reliability Controls

Correctness comes from short ACID transactions, database constraints, conditional writes, bounded classified DB retries, and idempotency. Liveness is process-only; readiness checks PostgreSQL and returns `503` when the API cannot commit. Controlled sold-out/voucher/idempotency/rate-limit outcomes use stable `4xx` contracts rather than being counted as infrastructure failures.

The process-local booking-attempt limiter defaults to 60 attempts per customer per 60

seconds and returns 429 with **Retry-After**. It reduces one instance's abuse/retry pressure but never replaces inventory or idempotency correctness. Multi-instance production requires a shared gateway/Redis policy.

12.3 Observability

The API validates, creates, and propagates **X-Request-ID**; responses and standard errors expose the same correlation value. One-line JSON logs include bounded request/event fields and omit request bodies, access tokens, voucher codes, and payment data.

GET /metrics exposes low-cardinality Prometheus text counters including HTTP totals, booking attempts/successes/replays/failures, sold-out and inventory conflicts, voucher exhaustion, idempotency conflicts, and expired bookings. These counters are in-process and reset on restart.

12.4 Implementation Status and Evolution

Table 12.1: Security, reliability, and observability status

Capability	Status	Boundary / evolution
Ownership and operation RBAC	Implemented	Backed by seeded user roles and scoped queries
Signed authentication / identity provider	Future	Replace trusted header with verified claims
DTO/business validation and stable errors	Implemented	Central validation and sanitized exception filter
Request correlation and structured logs	Implemented	Local JSON output; forward to managed log storage later
Liveness/readiness and metrics endpoint	Implemented	Metrics storage is currently single-instance memory
Booking rate limiter	Implemented locally	Move key/state to gateway or Redis for multiple replicas
Distributed tracing and alerting	Designed, not implemented	Export OpenTelemetry/Prometheus and define SLO alerts
Secret manager and production TLS/WAF	Future	Deployment/platform responsibility outside submission

Testing Strategy and Correctness Evidence

13.1 Testing Layers

Table 13.1: Testing strategy

Layer	Purpose	Examples
Unit	Pure domain policies	Pricing, voucher eligibility, state machine, request fingerprint
Integration	Real PostgreSQL semantics	Transactions, constraints, repository SQL, rollback behavior
Concurrency	Prove critical invariants under races	Oversell, idempotency race, final voucher, confirm-vs-expire
API/Postman	Reviewer-visible workflow validation	Customer happy path, admin operations, documented edge cases
Load	Measure behavior under stated and burst traffic	Throughput, latency, expected 4xx vs system errors

13.2 Invariant-to-Test Evidence

Table 13.2: Critical correctness evidence

Test ID	Invariant	Scenario / Expected Result
CONC-001	INV-01	100 requests compete for 10 tickets; exactly 10 succeed; final stock=0
CONC-002	INV-02	20 concurrent requests use same user/key/payload; one logical booking exists
CONC-003	INV-03	20 users compete for final voucher quota=1; exactly one redemption succeeds
CONC-004	INV-05	Confirm and expiry race; exactly one valid final outcome and resource state
INT-001	INV-04	Voucher reservation failure causes ticket reservation to roll back
INT-002	INV-06	Changing current ticket price does not change existing booking total

13.3 Requirement Traceability

The traceability chain is **Requirement** → **Invariant** → **Design** → **Implementation** → **Test Evidence**. Representative critical paths are listed below; the endpoint and test appendices provide the complete reviewer-facing inventories.

Table 13.3: Critical requirement traceability

Req.	Invariant	Design	Implementation evidence	Test evidence
FR-05, NFR-01	INV-01	Conditional quantity update	<code>inventory-reservation.repository.ts</code> + DB checks	CONC-001
FR-20, NFR-02	INV-02	User/key/fingerprint claim and replay	<code>create-booking.service.ts</code> , <code>idempotency.repository.ts</code>	CONC-002
FR-06, NFR-03	INV-03	Conditional quota + active per-user uniqueness	<code>voucher-reservation.repository.ts</code> , migration 0004	CONC-003
FR-05, NFR-04	INV-04	One booking transaction	<code>create-booking.service.ts</code> , transaction helper	INT-001
FR-17-19	INV-05, INV-07	Conditional terminal transition + locked batch	lifecycle service/repository + expiry worker	CONC-004
FR-03, FR-05	INV-06	Server pricing and immutable snapshots	booking write repository + DB schema	INT-002

Req.	Invariant	Design	Implementation evidence	Test evidence
FR-11	INV-07, INV-08	Guarded transition and status history	operation service + lifecycle repository	Operation integration suite

13.4 Evidence Standard

Concurrency tests should verify both the HTTP/API result and final database state. A test that only counts response codes without checking inventory, booking rows, and voucher rows is insufficient evidence for consistency guarantees.

Performance and Flash-Sale Analysis

14.1 Expected Workload and Evaluation Goal

The stated peak of 300–500 booking requests per minute is approximately 5–8.3 requests per second. The evaluation therefore targets stable behavior at that rate and a short burst above it. It does not claim a universal capacity limit: flash-sale risk is dominated by hot-row contention, transaction length, and controlled conflict behavior rather than aggregate request volume alone.

14.2 Executed Environment

Table 14.1: Reproducible local load-test environment

Field	Value
Benchmark source commit	cfc291ac1ad3e6ac9ebb09d304db60cead07a62e
Host	Windows x64; Intel Core i5-12450H (8 cores / 12 logical processors); 15.6 GiB RAM
Runtime	Node.js v22.17.1; one API container
Database	PostgreSQL 16.14, one local Docker container
Load engine	grafana/k6:0.53.0 through Docker
Target	http://host.docker.internal:3300; RATE_LIMIT_BOOKING_MAX=1000 confirmed in API container
Dataset	Deterministic customer, concert <code>summer-festival-2026</code> , STANDARD inventory

14.3 Executed Scenarios

The k6 script ran 5 iterations/second for 30 seconds, followed by a three-stage burst that ramped to 15 iterations/second and back over 30 seconds. It generated unique idempotency keys so each iteration represented a new logical booking. The rate limit was raised only to prevent the resilience guard from obscuring booking/database behavior.

Hot inventory, final-voucher, sold-out, and lifecycle contention are covered by dedicated deterministic concurrency suites. They were not represented as separate latency/capacity scenarios in this k6 baseline; presenting them as measured load results would overstate the evidence.

14.4 Measured Results

Evidence Verified k6 baseline

Requests: 501 completed / 0 interrupted

Throughput: 8.35 requests/second (reported as 8 req/s in the committed summary)

Latency: p50 13.31 ms; p95 24.87 ms; p99 31.73 ms; maximum 47.63 ms

Booking outcomes: 501 successes; 0 controlled business rejects

System error rate: 0.000%

Thresholds: system error < 1%, p95 < 1000 ms, and p99 < 2000 ms all passed

The exact generation time, database build string, reproduction command, and summary are committed in `docs/PERFORMANCE_REPORT.md`. Controlled 400/409/429 outcomes are intentionally classified as business rejects, not server failures. This particular run had none because the seeded STANDARD capacity and limiter were sufficient for all 501 unique bookings.

14.5 Interpretation and Limitations

The measured baseline is comfortably inside the stated workload and configured thresholds on one local API/database pair. It demonstrates absence of 5xx or unexpected status under this profile, not production capacity, multi-instance behavior, or saturation. CPU allocation to Docker, database locality, connection pool limits, and the non-hot STANDARD category materially influence the result.

Before a production claim, execute longer steady-state and soak tests; isolate sold-out and single-hot-category latency; measure final-voucher contention; capture database connection/lock wait/CPU/I/O; and repeat with the intended deployment topology and

shared rate limiter.

14.6 Optimization Priorities

1. preserve short booking transactions and deterministic resource order;
2. monitor connection saturation, lock waits, and the expiry index before scaling API replicas;
3. keep controlled sold-out/voucher/rate-limit outcomes separate from 5xx;
4. avoid every external network call inside finite-resource transactions;
5. introduce admission control, partitioning, or queueing only after hot-row evidence justifies it.

Architecture Decisions and Trade-offs

15.1 Summary of Major Decisions

Table 15.1: Architecture trade-off summary

Decision	Chosen Option	Why / Trade-off
Service architecture	Modular monolith	Simple ACID transactions and fast assessment delivery; less independent scaling than microservices
Inventory consistency	PostgreSQL atomic update	Single source of truth and simple failure semantics; hot rows can contend
Retry safety	DB-backed idempotency	Correct across process restarts; adds storage/lifecycle semantics
Expiry processing	DB worker + optional SKIP LOCKED	No broker required; polling introduces bounded delay
Async evolution	Transactional outbox (optional)	Reliable bridge to future broker; adds worker/table complexity
Rate limiting	Optional resilience layer	Reduces abuse/retry storm; does not replace transactional correctness
Payment boundary	No provider in current scope	Keeps assessment focused; backend cannot accept or verify real payments

15.2 What Was Deliberately Not Chosen

15.2.1 Microservices

The stated traffic and assessment time frame do not justify distributed deployment, service discovery, inter-service retries, and cross-service transaction complexity for the core booking path.

15.2.2 Redis Distributed Lock for Inventory

Inventory is already authoritative in PostgreSQL. Adding a Redis lock would introduce a second coordination system and additional distributed failure modes without a demonstrated need.

15.2.3 Kafka as a Baseline Requirement

Kafka is useful when asynchronous integrations and independent consumers justify it. It is not required to make ticket reservation correct at the stated load. A transactional outbox documents a safe evolution path if event-driven integration becomes necessary.

15.3 Decision Review and Revisit Triggers

Each ADR records context, alternatives, decision, rationale, consequences, and a trigger for reconsideration. The decision is therefore a current fit, not a claim that the selected technology is universally superior.

Table 15.2: Operational triggers for architecture evolution

Decision	Revisit when
Modular monolith	A module needs independent ownership/deployment/scaling and its consistency boundary can be separated
PostgreSQL atomic inventory	Measured hot-row lock waits or write-region limits violate the target SLO
Database idempotency	Retention volume or cross-region replay requires a dedicated lifecycle/storage strategy
Polling expiry worker	Poll delay, scan cost, or worker volume requires scheduled broker delivery
Optional outbox	A real email/payment/analytics consumer needs guaranteed post-commit publication
Process-local iter/metrics	More than one API replica makes per-instance state operationally misleading

Implemented Features, Omitted Features, and Limitations

16.1 Implemented Scope Matrix

Table 16.1: Implementation scope

Capability	Status	Notes
Concert browsing	Implemented	Published customer browse/detail/categories
Ticket category and availability	Implemented	Informational snapshot; booking rechecks atomically
Create booking	Implemented	Server pricing, inventory, voucher and persistence are atomic
Idempotent retries	Implemented	User-scoped key, canonical fingerprint and replay
Voucher validation/quota	Implemented	Eligibility, quota race, redemption and release
Reservation expiry	Implemented	Polling worker with transactional release and SKIP LOCKED
Operation booking monitoring	Implemented	Filter/list/detail with role guard and safe metadata

Capability	Status	Notes
Manual guarded status transition	Implemented	State machine, resource release and audit history
Concert/category create and publish	Implemented	Draft creation, inventory-backed category, guarded publication
Voucher create API	Implemented	Percentage/fixed policy with time, quota, and optional scope
Concert/voucher update or delete	Not implemented	Historical/versioning semantics intentionally unspecified
Payment/refund settlement	Out of scope	Confirmation is simulated; provider integration is future work
Seat-level reservation	Out of scope	Current model reserves category capacity only
Refunds/chargebacks/reconciliation	Out of scope	Depend on a future payment integration
Email/SMS notification	Not implemented	Future post-commit/outbox consumer
Distributed rate limiting/metrics	Partial	Current process-local policy; production evolution is Redis/gateway + telemetry
Multi-region active-active	Out of scope	PostgreSQL is the single correctness boundary

16.2 Known Limitations and Evolution

The current identity header, simulated payment, category-level inventory, process-local rate limiter/metrics, and polling expiry worker are deliberate assessment trade-offs. Their operational impact and production evolution path are recorded in `docs/ASSUMPTIONS_SCOPE_LIMITATIONS.md`. None of these limitations relaxes the critical invariants: inventory cannot oversell, idempotent retries cannot double-book, voucher quota cannot over-redeem,

and expiry cannot double-release resources.

16.3 Feature Scope: Voucher Operations

16.3.1 Assumption

The assessment requires voucher behavior needed by the booking flow, not a full campaign-management product.

16.3.2 Implemented

Operators/admins can create validated percentage or fixed vouchers with a time window, global quota, and optional concert/category scope. Customers can apply them atomically during booking; expiry/cancellation can release reserved quota.

16.3.3 Not Implemented

Voucher update, disable-through-API, delete, bulk import, campaign analytics, and policy versioning endpoints are absent.

16.3.4 Reason, Consequence, and Production Evolution

Mutation/deletion after redemption needs explicit historical and authorization rules, so it was omitted rather than implemented ambiguously. The consequence is that an existing campaign cannot be administered through this API. A production extension should add immutable voucher versions, audit every administrative change, define disable semantics for existing reservations, and protect elevated commands with production identity and approval policy.

Payment Gateway Scope Decision

17.1 Current Assessment Decision

Decision Real payment processing is intentionally not implemented

The submitted backend focuses on finite ticket inventory, retry safety, voucher consistency, booking lifecycle, and flash-sale behavior. A newly created booking is a time-limited reservation. Confirmation is simulated through the owned customer command or an authorized operation workflow; it does not prove card, bank, wallet, or provider settlement.

Adding a provider SDK would introduce network failure, webhook authentication, settlement, refunds, reconciliation, and sensitive-data concerns without materially strengthening evidence for the assessment's core correctness risks. The omission is therefore a deliberate scope decision, not an accidental gap.

17.2 Implemented Payment-Adjacent Behavior

The current code supports the parts of booking lifecycle that a later payment orchestration module would depend on:

- create a ten-minute **RESERVED** booking and atomically hold tickets and optional voucher capacity;
- persist booking state, immutable totals, expiry time, and audit history;
- conditionally transition a live **RESERVED** booking to **CONFIRMED**;
- expire or cancel a reservation and release finite resources exactly once;
- reject a terminal-state race with a stable conflict;
- apply idempotency to booking creation.

17.3 Explicitly Not Implemented

There is no external payment-provider client, payment session, authorization or capture, card/wallet/bank handling, provider webhook verification, payment status table, settlement, refund, chargeback, reconciliation, PCI-DSS card-data handling, or payment-specific idempotency workflow. The backend cannot accept or verify real customer payments.

17.4 Why Production Uses an External Gateway

A production system normally delegates sensitive payment execution to a specialized external gateway. The backend still owns payment orchestration, the booking/payment association, persisted payment state, verified and idempotent callback processing, booking transitions, expiry rules, and reconciliation jobs. The provider owns the customer payment interaction and regulated processing surface.

Figure 17.1 is an evolution model only; none of its dashed provider interactions exists in the submitted code.

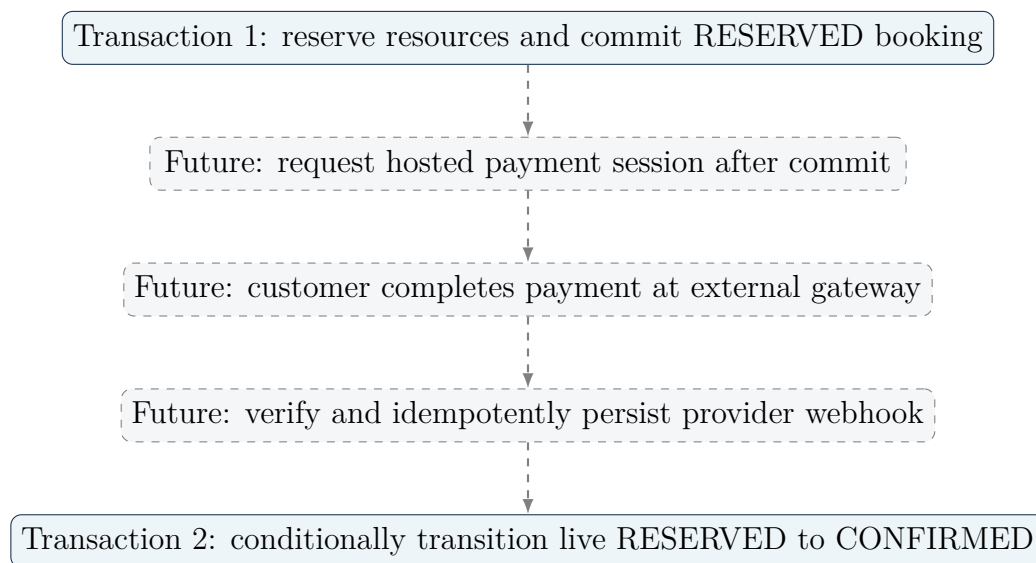


Figure 17.1: Future external payment flow (not implemented)

If payment completes after expiry wins, the backend must not resurrect the terminal booking. A production policy would record the late provider result and start a refund/compensation or manual reconciliation path.

17.5 Transaction Design Rule

The ticket reservation transaction must never remain open while waiting on a provider network call. External systems cannot participate in the same local ACID transaction; holding row locks across network latency amplifies hot-row contention and makes provider timeouts database failures.

Table 17.1: Payment transaction boundary

Phase	Preferred	Avoid
Reservation	Reserve inventory, create <code>RESERVED</code> , commit	Keep transaction open while calling provider
Provider interaction	Occur after commit with its own timeout/idempotency	Treat network response as part of local ACID
Result handling	Verify callback; begin a new transaction; conditionally confirm	Blindly overwrite an expired/cancelled booking
Failure recovery	TTL release plus reconciliation/compensation	Indefinite resource hold or untracked dual write

17.6 Limitation and Evolution Statement

The implementation does not address webhook signature/replay security, provider outage and retry policy, settlement mismatch, refund/chargeback workflow, or financial reconciliation. A production evolution should add a separate payment orchestration module and payment-attempt model, provider adapter, verified idempotent webhook inbox, explicit payment state machine, transactional outbox for side effects, reconciliation jobs, and operational alerts—without changing PostgreSQL’s authority over booking state.

Conclusion

The assessment backend is designed around explicit, traceable guarantees rather than a large number of loosely specified features. PostgreSQL provides the consistency boundary for ticket inventory, voucher quota, booking state, audit history, and idempotency. The modular monolith keeps the critical booking transaction understandable and testable, while scalable row claiming and the optional transactional outbox provide credible evolution paths without being prerequisites for current correctness.

The report connects requirements to business rules, invariants, design choices, implementation evidence, and automated tests. Concurrency races, rollback paths, idempotent retries, lifecycle arbitration, CI smoke, and the measured k6 baseline provide reproducible evidence rather than prospective claims. OpenAPI, Postman, migrations, seed data, runbooks, and ADRs support independent reviewer verification.

The system's limitations are equally explicit. It does not process payments, select seats, send notifications, administer full catalog/voucher lifecycles, or claim multi-region production readiness. Those boundaries preserve an honest assessment result: the implemented workflows are internally consistent and defensible, while future payment and asynchronous integrations have a safe post-commit path that does not weaken the finite-resource transaction.

Local Setup and Runbook

A.1 Prerequisites

The final repository uses Node.js 22 LTS, npm, Docker Desktop with the Linux engine, and PostgreSQL 16 through Docker Compose. The pinned runtime is in `.nvmrc`; dependencies are installed from the committed lockfile.

A.2 Quick Start

```
1 cp .env.example .env
2 npm ci
3 npm run db:migrate
4 npm run db:seed
5 npm run start:dev
```

Listing A.1: Fresh local setup

For the fully reproducible API + database workflow:

```
1 docker compose up --build
2 curl http://localhost:3000/health
```

Listing A.2: Docker Compose setup

If host ports are occupied, set `API_HOST_PORT` and `POSTGRES_HOST_PORT` in `.env`. The API remains port 3000 inside the Compose network.

A.3 Seeded local identities

Fresh databases use stable development identities:

- customer: 00000000-0000-4000-8000-0000000000001;
- operator: 00000000-0000-4000-8000-0000000000002;
- concert slug: `summer-festival-2026`;
- standard category: 00000000-0000-4000-8000-0000000000102.

The identity header is a documented development simplification, not a real access token. Reset an existing local volume once after upgrading seed IDs: `npm run db:reset`.

A.4 Verification

- Health: GET `/health/live` (process liveness) and GET `/health/ready` (HTTP 200 when PostgreSQL is reachable);
- OpenAPI: GET `/openapi.yaml`;
- Metrics: GET `/metrics`;
- Postman: import the environment at `postman/local.environment.json` and one of the customer/operation collections.

A.5 Test Commands

```
1 npm run format:check
2 npm run lint
3 npm run typecheck
4 npm run test:unit
5 npm run db:migrate && npm run db:seed
6 npm run test:integration
7 npm run test:concurrency
8 npm run test:load
```

Listing A.3: Final quality gates

The load command requires k6 or Docker and writes the reproducible summary to `docs/PERFORMANCE_REPORT.md`. See `docs/TEST_STRATEGY.md` for the invariant-to-test mapping.

API Endpoint Matrix

Table B.1: Implemented API endpoint matrix

Method	Endpoint	Actor	Notes
GET	/health	Public	Database health
GET	/health/live	Public	Process liveness; no database dependency
GET	/health/ready	Public	PostgreSQL readiness
GET	/metrics	Public	Prometheus text counters
GET	/openapi.yaml	Public	Machine-readable API contract
GET	/api/concerts	Customer	Published concerts, pagination
GET	/api/concerts/{id}	Customer	UUID or slug detail
GET	/api/concerts/{id}/ticket-categories	Customer	Price + informational availability
POST	/api/bookings	Customer	Idempotency-Key; atomic reservation
GET	/api/me/bookings	Customer	Ownership-scoped history
GET	/api/bookings/{id}	Customer	Ownership-scoped detail
POST	/api/bookings/{id}/confirm	Customer	Conditional lifecycle transition
POST	/api/bookings/{id}/cancel	Customer	Conditional release transition
GET	/api/admin/bookings	Operator / Admin	Filter and paginate
GET	/api/admin/bookings/{id}	Operator / Admin	Investigation detail + audit
PATCH	/api/admin/bookings/{id}/status	Operator / Admin	Guarded manual transition
POST	/api/admin/concerts	Operator / Admin	Create draft concert
POST	/api/admin/concerts/{id}/ticket-categories	Operator / Admin	Create inventory-backed category
POST	/api/admin/concerts/{id}/publish	Operator / Admin	Publish valid future concert
POST	/api/admin/vouchers	Operator / Admin	Create scoped promotion

Health, metrics, and OpenAPI routes are intentionally outside the application /api prefix.

Test Evidence Summary

C.1 Evidence Capture

The final verification used PostgreSQL 16, Node.js 22 and the benchmark commit recorded in `docs/PERFORMANCE_REPORT.md`. Commands and interpretation rules are kept in `docs/TEST_STRATEGY.md`; raw machine-specific output is ignored.

Table C.1: Critical automated test evidence

Test ID	Scenario	Result	Final-state assertion
CONC-001	100 requests / 10 tickets	PASS	10 bookings, 90 controlled conflicts, inventory 0
CONC-002	20 same idempotency requests	PASS	One logical booking and one inventory decrement
CONC-003	20 users / last voucher	PASS	Exactly one redemption and quota never exceeds limit
CONC-004	Confirm vs expiry race	PASS	One terminal outcome and no double release
INT-001	Voucher failure rollback	PASS	Ticket inventory and voucher quota unchanged
INT-002	Historical price immutability	PASS	Existing booking snapshot unchanged after catalog update

The latest automated run passed 12 unit suites (32 tests), 7 integration suites (22 tests), and 4 concurrency suites (7 tests). The load baseline is 501 requests at the stated steady/burst

profile; p50/p95/p99 and system error rate are recorded in `docs/PERFORMANCE_REPORT.md`.

Architecture Decision Records

D.1 ADR-001 — Modular Monolith

Status: Accepted

Context: The critical booking path needs atomic consistency across inventory, voucher, and booking state.

Decision: Use a modular monolith for the assessment.

Alternatives: Microservices.

Consequences: Simpler transactions and local setup; lower independent-service scalability and deployment autonomy.

D.2 ADR-002 — PostgreSQL for Inventory Consistency

Status: Accepted

Decision: Use atomic conditional SQL updates and DB constraints rather than an external distributed lock as the primary correctness mechanism.

Consequence: Hot inventory rows may become contention points, but the failure model remains simple and explicit.

D.3 ADR-003 — Atomic Conditional Inventory Update

Status: Accepted

Decision: Reserve inventory with one `UPDATE ... WHERE available_quantity >= quantity RETURNING` statement inside the booking transaction.

Alternatives: Read-then-update, explicit row lock, optimistic versioning, Redis lock,

queue serialization.

Consequence: PostgreSQL remains authoritative and oversell is prevented; highly demanded rows may contend.

D.4 ADR-004 — Idempotent Booking API

Status: Accepted

Decision: Scope the idempotency key by customer, persist the request fingerprint and original result, and replay the same logical booking.

Alternatives: Process-local memory or cache-only deduplication.

Consequence: Correctness survives process restarts; idempotency records need a retention policy.

D.5 ADR-005 — Database-Polling Expiry Worker

Status: Accepted

Decision: Process expired reservations in batches with `FOR UPDATE SKIP LOCKED`; release inventory and voucher quota in the same transaction.

Consequence: Expiration timing is bounded by poll interval rather than instantaneous.

D.6 ADR-006 — Transactional Outbox (Optional)

Status: Proposed / Optional

Decision: If asynchronous integrations are implemented, write domain events to an outbox in the same business transaction.

Consequence: Adds outbox storage and a worker, but avoids DB+broker dual-write inconsistency.